Resolución — Taller SQL sobre HR 3 — Subconsultas y retos de RR. HH.

Bases de Datos I · resolución paso a paso

Antes de empezar

- Cada ejercicio muestra lo que se pide, la idea, los pasos, la consulta, su resultado en HR y un aviso de error común.
- Las consultas están escritas para Oracle Database 19c y solo leen el esquema HR. Los resultados salen de ejecutarlas ahí.
- Hay más de una forma correcta de resolver casi todos los ejercicios; cuando conviene, se muestra una variante.

Ejercicio 1 — Por encima del promedio de la empresa

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Mostrar los empleados que ganan más que el salario promedio de toda la empresa.

Mostrar: nombre, apellido y salario, del mayor al menor y, si empatan, por apellido.

Idea. Una subconsulta escalar calcula primero un número (el promedio) y la consulta externa lo usa para filtrar.

Pasos.

1. La subconsulta `(SELECT AVG(salary) FROM hr.employees)` devuelve un único valor.
2. La consulta externa compara `e.salary` contra ese valor.
3. Se ordena por salario de mayor a menor y, en empate, por apellido.

```
SELECT e.first_name,
       e.last_name,
       e.salary
FROM hr.employees e
WHERE e.salary > (SELECT AVG(x.salary) FROM hr.employees x)
ORDER BY e.salary DESC, e.last_name;
```

Resultado en HR. 51 filas y 3 columnas; se muestran las primeras 6 filas:

```
FIRST_NAME  LAST_NAME  SALARY
----------  ---------  ------
Steven      King        24000
Lex         Garcia      17000
Neena       Yang        17000
John        Singh       14000
Karen       Partners    13500
Michael     Martinez    13000
```

Ojo. La subconsulta va entre paréntesis y se ejecuta una sola vez, porque no depende de la fila de la consulta externa.

Ejercicio 2 — Mejor pagados que su departamento

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Mostrar todos los empleados cuyo salario sea mayor al promedio de su departamento.

Mostrar: nombre completo del empleado, salario y promedio del departamento (dos decimales), ordenado por departamento y, dentro de cada uno, de mayor a menor salario.

Idea. Cada empleado se compara con el promedio de SU departamento, no con uno global: el promedio se calcula por grupo y se une de vuelta a cada empleado.

Pasos.

1. La vista en línea p calcula un promedio por `department_id`.
2. Se unen `hr.employees` y p por `department_id`: cada empleado queda junto al promedio de su departamento.
3. El filtro `e.salary > p.promedio` conserva a quienes lo superan.
4. ROUND se aplica solo al mostrar, para no comparar contra un valor redondeado.

```
SELECT e.first_name || ' ' || e.last_name AS nombre_empleado,
       e.salary,
       ROUND(p.promedio, 2) AS promedio_departamento
FROM hr.employees e
JOIN (SELECT x.department_id,
             AVG(x.salary) AS promedio
      FROM hr.employees x
      GROUP BY x.department_id) p
  ON p.department_id = e.department_id
WHERE e.salary > p.promedio
ORDER BY e.department_id, e.salary DESC, e.last_name;
```

Resultado en HR. 38 filas y 3 columnas; se muestran las primeras 6 filas:

```
NOMBRE_EMPLEADO   SALARY  PROMEDIO_DEPARTAMENTO
---------------  ------  --------------------
Michael Martinez   13000                  9500
Den Li             11000                  4150
Adam Fripp          8200               3475.56
Matthew Weiss       8000               3475.56
Payam Kaufling      7900               3475.56
Shanta Vollman      6500               3475.56
```

Ojo. Quien no tiene departamento queda fuera: `NULL = NULL` no es verdadero en el `JOIN`.

Variante. Con una subconsulta correlacionada en el `WHERE` (sin mostrar el promedio). Devuelve las mismas 38 personas:

```
SELECT e.first_name, e.last_name, e.salary
FROM hr.employees e
WHERE e.salary > (SELECT AVG(x.salary)
                  FROM hr.employees x
                  WHERE x.department_id = e.department_id)
ORDER BY e.department_id, e.salary DESC, e.last_name;
```

Ejercicio 3 — El máximo de cada departamento

Nivel: Avanzado · **Tablas:** `employees`, `departments`

Se pide. Para cada departamento, mostrar al empleado (o empleados) que ganan el salario máximo de ese departamento.

Mostrar: departamento, nombre, apellido y salario, ordenado por departamento y apellido.

Idea. Una subconsulta correlacionada se vuelve a ejecutar para cada fila de la consulta externa porque usa un valor de esa fila.

Pasos.

1. Se unen `hr.employees` y `hr.departments` para leer el nombre del departamento.

2. La subconsulta calcula `MAX(x.salary)` de los empleados con el mismo `department_id` que la fila externa (`x.department_id = e.department_id`).
3. El filtro `e.salary = (subconsulta)` deja a quien gana ese máximo.
4. Se ordena por departamento y apellido.

```
SELECT d.department_name,
       e.first_name,
       e.last_name,
       e.salary
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
WHERE e.salary = (SELECT MAX(x.salary)
                  FROM hr.employees x
                  WHERE x.department_id = e.department_id)
ORDER BY d.department_name, e.last_name;
```

Resultado en HR. 11 filas y 4 columnas; se muestran las primeras 6 filas:

```
DEPARTMENT_NAME  FIRST_NAME  LAST_NAME  SALARY
---------------  ----------  ---------  ------
Accounting       Shelley     Higgins     12008
Administration   Jennifer    Whalen       4400
Executive        Steven      King        24000
Finance          Nancy       Gruenberg   12008
Human Resources  Susan       Jacobs       6500
IT               Alexander   James        9000
```

Ojo. Si dos empleados empatan en el máximo salen los dos: la igualdad no elige uno solo. En HR nadie empata en el máximo de su departamento: salen 11 filas, una por departamento.

Ejercicio 4 — ¿Quiénes son jefes?

Nivel: Intermedio · **Tablas:** `employees`, `jobs`

Se pide. Listar los empleados que son jefes de al menos una persona.

Mostrar: nombre, apellido y cargo (`job_title`), ordenados por apellido y nombre.

Idea. `EXISTS` pregunta si la subconsulta devuelve alguna fila; no importa qué columnas selecciona, por eso se escribe `SELECT 1`.

Pasos.

1. Un empleado es jefe si su `employee_id` aparece como `manager_id` de otro empleado.
2. `EXISTS (SELECT 1 FROM hr.employees s WHERE s.manager_id = e.employee_id)` busca al menos un subordinado.
3. Se une `hr.jobs` para mostrar el título del cargo.

```
SELECT e.first_name,
       e.last_name,
       j.job_title
FROM hr.employees e
JOIN hr.jobs j ON j.job_id = e.job_id
WHERE EXISTS (SELECT 1
              FROM hr.employees s
              WHERE s.manager_id = e.employee_id)
ORDER BY e.last_name, e.first_name;
```

Resultado en HR. 18 filas y 3 columnas; se muestran las primeras 6 filas:

```
FIRST_NAME  LAST_NAME  JOB_TITLE
----------  ---------  ----------------------------
Gerald      Cambrault  Sales Manager
Alberto     Errazuriz  Sales Manager
Adam        Fripp      Stock Manager
Lex         Garcia     Administration Vice President
Nancy       Gruenberg  Finance Manager
Shelley     Higgins    Accounting Manager
```

Ojo. `EXISTS` se detiene en la primera coincidencia; no cuenta subordinados. Si necesitas saber cuántos son, usa un `JOIN` con `COUNT`.

Ejercicio 5 — Sin subordinados y buen sueldo

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Listar los empleados que no son jefes de nadie y que ganan más de 10000.

Mostrar: nombre, apellido y salario, del mayor al menor.

Idea. «No es jefe de nadie» es la negación del ejercicio anterior: `NOT EXISTS`.

Pasos.

1. `NOT EXISTS (SELECT 1 FROM hr.employees s WHERE s.manager_id = e.employee_id)` deja a quien nadie tiene como jefe.
2. Se agrega `e.salary > 10000` con `AND`.
3. Se ordena por salario descendente.

```
SELECT e.first_name,
       e.last_name,
       e.salary
FROM hr.employees e
WHERE e.salary > 10000
  AND NOT EXISTS (SELECT 1
                  FROM hr.employees s
                  WHERE s.manager_id = e.employee_id)
ORDER BY e.salary DESC;
```

Resultado en HR. 3 filas y 3 columnas:

```
FIRST_NAME  LAST_NAME  SALARY
----------  ---------  ------
Lisa        Ozer        11500
Ellen       Abel        11000
Clara       Vishney     10500
```

Ojo. La versión con `NOT IN (SELECT manager_id FROM hr.employees)` devuelve cero filas: `manager_id` contiene un `NULL` (el empleado sin jefe) y `x NOT IN (..., NULL)` nunca es verdadero. `NOT EXISTS` no tiene ese problema.

Ejercicio 6 — Candidatos que cumplen todo

Nivel: Reto · **Tablas:** `employees`

Se pide. Una empresa quiere identificar a los empleados que cumplan todas estas condiciones: (a) su salario es superior al promedio de su departamento; (b) fueron contratados después del empleado más antiguo de su departamento; (c) no tienen subordinados directos, es decir, ningún empleado los tiene como jefe; (d) su salario es inferior al salario máximo de su departamento.

Mostrar: todas las columnas del empleado, ordenadas por `employee_id`.

Idea. Cada condición es una pregunta distinta y todas se unen con AND. Las tres que miran al departamento son subconsultas correlacionadas; la de los subordinados es un NOT EXISTS.

Pasos.

1. Se parte de hr.employees e, el empleado que se evalúa.
2. Condición (a): e.salary > (SELECT AVG(x.salary) ...) sobre los empleados de su mismo departamento.
3. Condición (b): e.hire_date > (SELECT MIN(x.hire_date) ...), la fecha del más antiguo de su departamento.
4. Condición (c): NOT EXISTS de empleados cuyo manager_id sea el employee_id del evaluado.
5. Condición (d): e.salary < (SELECT MAX(x.salary) ...) sobre su departamento.
6. e.* proyecta todas las columnas del empleado, y ORDER BY e.employee_id las ordena por identificador.

```
SELECT e.*
FROM hr.employees e
WHERE e.salary > (SELECT AVG(x.salary)
                  FROM hr.employees x
                  WHERE x.department_id = e.department_id)
  AND e.hire_date > (SELECT MIN(x.hire_date)
                     FROM hr.employees x
                     WHERE x.department_id = e.department_id)
  AND NOT EXISTS (SELECT 1
                  FROM hr.employees s
                  WHERE s.manager_id = e.employee_id)
  AND e.salary < (SELECT MAX(x.salary)
                  FROM hr.employees x
                  WHERE x.department_id = e.department_id)
ORDER BY e.employee_id;
```

Resultado en HR. 20 filas y 11 columnas; se muestran las primeras 6 filas y 6 de las 11 columnas:

```
EMPLOYEE_ID  FIRST_NAME  LAST_NAME  EMAIL     PHONE_NUMBER    HIRE_DATE
-----------  ----------  ---------  --------  --------------  ---------
        104  Bruce       Miller     BMILLER   1.590.555.0104  21-MAY-17
        137  Renske      Ladwig     RLADWIG   1.650.555.0137  14-JUL-13
        141  Trenna      Rajs       TRAJS     1.650.555.0141  17-OCT-13
        150  Sean        Tucker     STUCKER   44.1632.960005  30-JAN-15
        151  David       Bernstein  DBERNSTE  44.1632.960006  24-MAR-15
        152  Peter       Hall       PHALL     44.1632.960007  20-AUG-15
```

Ojo. Los empleados sin departamento se descartan: x.department_id = NULL nunca es verdadero. Traduce cada condición del enunciado de forma literal y léelas una por una antes de unirlas.

Variante. La misma lógica en una sola pasada por departamento: una vista en línea calcula promedio, fecha más antigua y máximo, y el JOIN los reparte a cada empleado. Devuelve los mismos 20 empleados.

```
SELECT e.*
FROM hr.employees e
JOIN (SELECT x.department_id,
             AVG(x.salary)    AS promedio,
             MIN(x.hire_date) AS mas_antiguo,
             MAX(x.salary)    AS maximo
      FROM hr.employees x
      GROUP BY x.department_id) d
  ON d.department_id = e.department_id
WHERE e.salary > d.promedio
  AND e.hire_date > d.mas_antiguo
  AND e.salary < d.maximo
  AND NOT EXISTS (SELECT 1
                  FROM hr.employees s
                  WHERE s.manager_id = e.employee_id)
ORDER BY e.employee_id;
```

Ejercicio 7 — El segundo salario más alto

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Mostrar quiénes cobran el segundo salario más alto de la empresa, entendido como el mayor valor que sea menor al máximo.

Mostrar: nombre, apellido y salario, ordenados por apellido.

Idea. Se calcula el máximo, luego el máximo entre los salarios menores a ese, y por último se busca a quienes cobran exactamente ese valor.

Pasos.

1. El salario más alto es `(SELECT MAX(y.salary) FROM hr.employees y)`.
2. El segundo es el `MAX` de los salarios que sean menores al primero.
3. La consulta externa trae a quienes ganan exactamente ese valor.

```
SELECT e.first_name,
       e.last_name,
       e.salary
FROM hr.employees e
WHERE e.salary = (SELECT MAX(x.salary)
                  FROM hr.employees x
                  WHERE x.salary < (SELECT MAX(y.salary)
                                    FROM hr.employees y))
ORDER BY e.last_name;
```

Resultado en HR. 2 filas y 3 columnas:

```
FIRST_NAME  LAST_NAME  SALARY
----------  ---------  ------
Lex         Garcia      17000
Neena       Yang        17000
```

Ojo. `ORDER BY salary DESC FETCH FIRST 2 ROWS ONLY` no sirve: devolvería a Steven King (24000) y solo a una de las dos personas que cobran 17000, y la primera fila ni siquiera es del segundo salario. Hablar del segundo valor distinto, que aquí comparten dos empleados, exige la subconsulta.

Ejercicio 8 — Departamentos que pagan por encima de la empresa

Nivel: Avanzado · **Tablas:** `employees`, `departments`

Se pide. Listar los departamentos cuyo salario promedio es mayor que el salario promedio de toda la empresa.

Mostrar: departamento y salario promedio (dos decimales), de mayor a menor.

Idea. Se compara el promedio de cada grupo contra un promedio global calculado en una subconsulta escalar.

Pasos.

1. Se agrupa por departamento y se calcula `AVG(e.salary)`.
2. El promedio global sale de `(SELECT AVG(salary) FROM hr.employees)`.
3. `HAVING AVG(e.salary) > (subconsulta)` compara cada grupo contra el global.
4. Se ordena por el promedio.

```
SELECT d.department_name,
       ROUND(AVG(e.salary), 2) AS promedio
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING AVG(e.salary) > (SELECT AVG(x.salary) FROM hr.employees x)
ORDER BY promedio DESC;
```

Resultado en HR. 7 filas y 2 columnas:

```
DEPARTMENT_NAME   PROMEDIO
---------------  --------
Executive         19333.33
Accounting           10154
Public Relations     10000
Marketing             9500
Sales              8955.88
Finance            8601.33
Human Resources       6500
```

Ojo. La comparación va en `HAVING` y no en `WHERE`, porque depende de una función de agregación. Y se compara con el promedio exacto, no con el redondeado.

Ejercicio 9 — Un cargo que ya habían tenido

Nivel: Avanzado · **Tablas:** `employees`, `jobs`, `job_history`

Se pide. Listar los empleados que hoy ocupan el mismo cargo que ya habían tenido en otro periodo registrado en el historial de cargos.

Mostrar: nombre, apellido y cargo actual (`job_title`), ordenados por apellido.

Idea. `EXISTS` sobre `job_history` pregunta si hay un registro del mismo empleado con el mismo cargo.

Pasos.

1. `hr.job_history` guarda los cargos anteriores de cada empleado (`employee_id`, `job_id`).
2. `EXISTS` busca una fila del historial del mismo empleado con el mismo `job_id` que su cargo actual.
3. Se une `hr.jobs` para escribir el título del cargo.

```
SELECT e.first_name,
       e.last_name,
       j.job_title
FROM hr.employees e
JOIN hr.jobs j ON j.job_id = e.job_id
WHERE EXISTS (SELECT 1
              FROM hr.job_history h
              WHERE h.employee_id = e.employee_id
                AND h.job_id      = e.job_id)
ORDER BY e.last_name;
```

Resultado en HR. 2 filas y 3 columnas:

```
FIRST_NAME  LAST_NAME  JOB_TITLE
----------  ---------  ------------------------
Jonathon    Taylor     Sales Representative
Jennifer    Whalen     Administration Assistant
```

Ojo. Puede devolver muy pocas filas: `job_history` solo tiene 10 registros en este esquema.

Ejercicio 10 — Por encima del promedio de su cargo

Nivel: Avanzado · **Tablas:** `employees`, `jobs`

Se pide. Mostrar los empleados que ganan más que el promedio de los empleados que tienen su mismo cargo.

Mostrar: nombre, apellido, cargo, salario y promedio del cargo (dos decimales), ordenados por cargo, salario descendente y, si empatan, apellido.

Idea. Es el mismo patrón del ejercicio 2, pero el grupo de comparación es el cargo (`job_id`) y no el departamento.

Pasos.

1. La vista en línea p calcula un promedio por `job_id`.
2. Se une `hr.employees` con p y con `hr.jobs` para leer el título.
3. El filtro `e.salary > p.promedio` deja a quienes superan el promedio de su cargo.
4. ROUND se aplica solo al mostrar.

```
SELECT e.first_name,
       e.last_name,
       j.job_title,
       e.salary,
       ROUND(p.promedio, 2) AS promedio_cargo
FROM hr.employees e
JOIN hr.jobs j ON j.job_id = e.job_id
JOIN (SELECT x.job_id,
             AVG(x.salary) AS promedio
      FROM hr.employees x
      GROUP BY x.job_id) p
  ON p.job_id = e.job_id
WHERE e.salary > p.promedio
ORDER BY j.job_title, e.salary DESC, e.last_name;
```

Resultado en HR. 43 filas y 5 columnas; se muestran las primeras 6 filas:

```
FIRST_NAME  LAST_NAME  JOB_TITLE         SALARY  PROMEDIO_CARGO
----------  ---------  ----------------  ------  --------------
Daniel      Faviet     Accountant          9000            7920
John        Chen       Accountant          8200            7920
Alexander   James      Programmer          9000            5760
Bruce       Miller     Programmer          6000            5760
Alexander   Khoo       Purchasing Clerk    3100            2780
Shelli      Baida      Purchasing Clerk    2900            2780
```

Ojo. Los cargos con un solo empleado nunca aparecen: nadie supera su propio promedio.

Ejercicio 11 — Más que todos los de Shipping

Nivel: Avanzado · **Tablas:** `employees`, `departments`

Se pide. Mostrar los empleados que ganan más que cualquiera de los empleados del departamento Shipping, es decir, más que todos ellos.

Mostrar: nombre, apellido y salario, del mayor al menor y, si empatan, por apellido.

Idea. `> ALL (subconsulta)` exige ser mayor que cada uno de los valores que devuelve la subconsulta.

Pasos.

1. La subconsulta trae los salarios del departamento cuyo nombre es `Shipping`.
2. `e.salary > ALL (...)` pide ser mayor que cada uno de ellos, o sea mayor que el máximo.
3. Se ordena de mayor a menor.

```
SELECT e.first_name,
       e.last_name,
       e.salary
FROM hr.employees e
WHERE e.salary > ALL (SELECT x.salary
                      FROM hr.employees x
                      JOIN hr.departments dx
                        ON dx.department_id = x.department_id
                      WHERE dx.department_name = 'Shipping')
ORDER BY e.salary DESC, e.last_name;
```

Resultado en HR. 31 filas y 3 columnas; se muestran las primeras 6 filas:

```
FIRST_NAME  LAST_NAME  SALARY
----------  ---------  ------
Steven      King        24000
Lex         Garcia      17000
Neena       Yang        17000
John        Singh       14000
Karen       Partners    13500
Michael     Martinez    13000
```

Ojo. Si la subconsulta devolviera algún `NULL`, `> ALL` nunca sería verdadero. Y sobre un conjunto vacío es verdadero para todas las filas.

Variante. `> ANY` equivale a «mayor que el mínimo» y `> ALL` a «mayor que el máximo». Por eso este ejercicio también se resuelve con `e.salary > (SELECT MAX(...))`: el máximo de Shipping es 8200 y las dos formas devuelven las mismas 31 filas.

Ejercicio 12 — El mejor pagado de cada país

Nivel: Reto · **Tablas:** `employees`, `departments`, `locations`, `countries`

Se pide. Para cada país con empleados, mostrar quién tiene el salario más alto de ese país. Si hay empate, mostrar a todos los empatados.

Mostrar: país, nombre, apellido y salario, ordenado por país.

Idea. El país se alcanza viajando por cuatro tablas, y la subconsulta correlacionada repite ese viaje para calcular el máximo de cada país.

Pasos.

1. El país se alcanza por `employees`, `departments`, `locations` y `countries`.
2. La subconsulta repite ese mismo camino con otros alias (`x`, `dx`, `lx`) y calcula el máximo salario del mismo `country_id` que la fila externa (`lx.country_id = l.country_id`).
3. El filtro `e.salary = (subconsulta)` deja al mejor pagado de cada país.
4. Se ordena por país.

```
SELECT c.country_name,
       e.first_name,
       e.last_name,
       e.salary
FROM hr.employees   e
JOIN hr.departments d ON d.department_id = e.department_id
JOIN hr.locations   l ON l.location_id   = d.location_id
JOIN hr.countries   c ON c.country_id    = l.country_id
WHERE e.salary = (SELECT MAX(x.salary)
                  FROM hr.employees   x
                  JOIN hr.departments dx
                    ON dx.department_id = x.department_id
                  JOIN hr.locations   lx
                    ON lx.location_id = dx.location_id
                  WHERE lx.country_id = l.country_id)
ORDER BY c.country_name, e.last_name;
```

Resultado en HR. 4 filas y 4 columnas:

```
COUNTRY_NAME                  FIRST_NAME  LAST_NAME  SALARY
----------------------------  ----------  ---------  ------
Canada                        Michael     Martinez    13000
Germany                       Hermann     Brown       10000
United Kingdom of Great Bri...  John        Singh       14000
United States of America      Steven      King        24000
```

Ojo. Usar alias distintos en la subconsulta (`x`, `dx`, `lx`) evita confundirla con las tablas de afuera. Es la forma explícita; con funciones de ventana quedaría más corta.